



Graphs

A Rigorous, Ground-Up Guide to Graph Theory
and Implementation in the C Programming Language

From Adjacency Lists to Topological Sort — Memory-Safe, Pointer-Precise,
Production-Ready

2026 ACM Handout Series

100 Days Of Code

Abstract: This guide presents a complete, academically rigorous treatment of graph data structures implemented in C. It covers the theoretical foundations of graphs as abstract data types, their two canonical physical representations (adjacency matrices and adjacency lists),

and the memory-management discipline required to implement them safely in a language with manual allocation. Readers will progress from motivating why graphs are necessary, through core traversal algorithms (BFS, DFS), algorithmic patterns (topological sort, cycle detection), common bugs and their fixes, competitive-programming considerations, and concluding with full implementations of Dijkstra's, Prim's, and Kruskal's algorithms. Every code listing is written in ISO C (C11), designed to compile with `gcc -Wall -Wextra`, and annotated with complexity analysis.

Contents

S.No	Topic	Page
1	Title Page	—
2	Table of Contents	—
3	The Problem — Why Graphs Exist	i
4	Trade-off Comparison Table	i
5	When to Use Graphs (Status & Industry)	i
6	Core Building Blocks (Matrix & List Structs)	ii
7	The Critical Concept — The visited Array	iii
8	Glossary of Terms	iv
9	Cross-Language Syntax Comparison	v
10	Analogies	v
11	Static ASCII Diagrams	v
12	BFS Step-by-Step Walkthrough	vi
13	Complexity Intuition — $O(V+E)$ vs $O(V^2)$	vi
14	Complexity Reference Table	vii
15	Golden Rules	vii
16	Algorithmic Patterns (Topo Sort, Cycle Detection)	vii
17	Full Implementation — Adjacency List Graph	x
18	Bug Scenarios & Fixes	xii
19	Graph Variants (DAG, Weighted, Bipartite)	xiv
20	Deep Theory — Memory Layout & Cache Locality	xv
21	Memory Model — Stack vs Heap	xv
22	Problem Ladder	xvi
23	Hints	xvi
24	Complexity Table (Reference Copy)	xvii
25	Pattern Cheat Sheet	xvii
26	Competitive Programming Platform Tips	xvii
27	Common Mistakes — CP Checklist	xviii
28	Extended Debugging (Self-loops, Isolated Vertices)	xviii
29	What You've Learned	xix
30	Forward-Looking Statement	xx
31	Weighted Graph Algorithms — Dijkstra, Prim's, Kruskal's	xx

1 The Problem: What Trees and Arrays Cannot Do

Arrays give you $O(1)$ index access but only linear, positional relationships between elements. Trees extend this with hierarchy — a strict parent-to-many-children relationship — but a tree fundamentally forbids two structural realities that show up constantly in real systems:

1. **Cycles.** A tree, by definition, is acyclic and connected with exactly one path between any two nodes. But a road network has loops (you can drive back to where you started). A dependency graph can — accidentally or intentionally — have circular references that must be *detected*, not merely disallowed by the data structure.
2. **Many-to-many relationships.** In a tree, a node has exactly one parent. But a student can enroll in many courses, and a course can have many students. A city can be reached from many other cities directly, not just from a single “parent” city. This is an inherently many-to-many topology, and trees cannot express it without artificial duplication.

A graph $G = (V, E)$ generalizes both array and tree structures: it is a set of vertices V and a set of edges E connecting pairs of vertices, with **no constraint** on how many edges touch a vertex, and **no constraint** against cycles. This makes the graph the correct abstraction any time relationships are arbitrary, bidirectional, or cyclic — which, empirically, is most of the time in real-world systems.

2 Trade-off Comparison

Dimension	Array	Tree	Graph
Relationship type	Positional/linear	Parent $\rightarrow$ children	Many-to-many, arbitrary
Cycles allowed	N/A	No (by def.)	Yes
Traversal	$O(n)$	$O(n)$	$O(V + E)$ list / $O(V^2)$ matrix
Natural use case	Sequential data	Hierarchies (file sys., DOM)	Networks (roads, social)
Lookup by key	$O(1)$ by index	$O(\log n)$ balanced	$O(1)$ matrix / $O(\text{degree})$ list
Memory overhead	Minimal	Moderate	Moderate–high
Insertion	$O(n)$ worst	$O(\log n)$ balanced	$O(1)$ list / $O(1)$ matrix
Cycle detection?	No	No	Yes — active concern

Table 1: Array vs Tree vs Graph — feature comparison

3 When to Use Graphs — Status & Industry

Graphs are not a niche academic tool; they are the standard representation for interconnected systems across nearly every major industry:

- **Logistics & routing** — UPS, FedEx, and ride-share platforms model road networks as weighted graphs and run shortest-path algorithms (Dijkstra, A*) for real-time routing.

- **Social networks** — Facebook, LinkedIn, and X model users as vertices and relationships as edges to compute friend suggestions, community detection, and influence propagation.
- **Hospital & healthcare records** — Patient-provider-treatment relationships, drug interaction networks, and epidemiological contact-tracing systems are modeled as graphs.
- **Academia & citation networks** — Google Scholar and citation-impact tools represent papers as vertices and citations as directed edges.
- **Compilers & build systems** — Makefiles, npm/cargo dependency resolution, and compiler instruction scheduling all rely on directed acyclic graphs (DAGs) and topological sorting.
- **Networking & telecom** — Routers use graph algorithms (OSPF uses Dijkstra’s algorithm directly) to compute optimal packet-forwarding paths.
- **Finance** — Fraud-detection systems model transactions as a graph to detect suspicious cyclic money flows.

Decision Checklist — Use a Graph When:

- ☐ Entities have relationships that are not strictly hierarchical.
- ☐ A relationship needs to be many-to-many (not one-to-many).
- ☐ Cycles are possible or must be explicitly detected/prevented.
- ☐ You need shortest-path, connectivity, or reachability queries.
- ☐ Weighted or directional relationships matter (distance, cost, dependency order).
- ☐ The domain is naturally described as “a network of X.”

4 Core Building Blocks

4.1 Adjacency Matrix — Minimal C Structure

```
1  #define MAX_V 100
2
3  typedef struct {
4      int V;                /* number of vertices */
5      int adj[MAX_V][MAX_V]; /* adj[i][j] = 1 if edge i→j exists, else 0 */
6  } GraphMatrix;
7
8  void initGraphMatrix(GraphMatrix *g, int vertices) {
9      g→V = vertices;
```

```

10     for (int i = 0; i < vertices; i++)
11         for (int j = 0; j < vertices; j++)
12             g->adj[i][j] = 0;
13 }

```

4.2 Adjacency List — Minimal C Structure

```

1  typedef struct Node {
2      int dest;           /* destination vertex of this edge */
3      struct Node *next; /* next edge in this vertex's list */
4  } Node;
5
6  typedef struct {
7      int V;              /* number of vertices */
8      Node **adjList;     /* array of linked-list heads, one per vertex */
9  } GraphList;

```

Text-based visual diagram of an adjacency list for vertices $\{0,1,2,3\}$ with edges $0 \rightarrow 1$, $0 \rightarrow 2$, $1 \rightarrow 3$:

```

adjList[0] → [1] → [2] → NULL
adjList[1] → [3] → NULL
adjList[2] → NULL
adjList[3] → NULL

```

5 The Critical Concept — Tracking Visited State

The single most common source of bugs in graph code is mismanaging the **visited** array during traversal. Without it, cyclic graphs cause **infinite loops** because the algorithm keeps revisiting the same vertices forever.

WRONG — No Visited Tracking (Infinite Loop on Any Cycle)

```

1  void dfsWrong(GraphList *g, int v) {
2      printf("%d ", v);
3      Node *cur = g->adjList[v];
4      while (cur != NULL) {
5          dfsWrong(g, cur->dest); /* BUG: may revisit v forever if a cycle exists */
6          cur = cur->next;
7      }
8  }

```

CORRECT — Visited Array Marks Nodes Before Recursing

```

1 void dfsCorrect(GraphList *g, int v, int *visited) {
2     visited[v] = 1;           /* mark BEFORE exploring neighbors */
3     printf("%d ", v);
4     Node *cur = g->adjList[v];
5     while (cur != NULL) {
6         if (!visited[cur->dest]) {
7             dfsCorrect(g, cur->dest, visited);
8         }
9         cur = cur->next;
10    }
11 }

```

Warning

Key discipline: Mark a vertex visited the moment you decide to process it — not after you finish processing it. Marking too late allows the same vertex to be queued or recursed into multiple times before the first visit completes.

6 Glossary of Terms

Term	Definition
Vertex (Node)	A fundamental unit of the graph representing an entity
Edge	A connection between two vertices, may be directed or undirected
Degree	Number of edges incident to a vertex (in-degree + out-degree for directed graphs)
Path	A sequence of vertices connected by edges, no repeated vertices
Cycle	A path that starts and ends at the same vertex
Directed	Edges have a direction ($A \rightarrow B$ does not imply $B \rightarrow A$)
Undirected	Edges are bidirectional ($A - B$ implies $B - A$)
Bipartite	Vertices can be split into two sets such that every edge connects the two sets
Weighted	Each edge carries a numeric cost/distance/capacity
Connected Component	A maximal set of vertices all reachable from one another
Adjacency	Two vertices are adjacent if an edge connects them directly

Table 2: Graph theory glossary

7 Cross-Language Syntax Comparison

Language	Graph Declaration
C	Node **adjList; (array of linked lists)
Python	graph = {0: [1, 2], 1: [3], 2: [], 3: []}
C++	std::vector<std::vector<int>> adj(V);

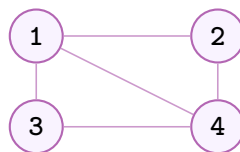
C requires the programmer to manage the linked-list nodes manually with `malloc/free`; Python and C++ handle the underlying dynamic array/list resizing automatically. This is the central theme of this guide: C gives full control over memory layout at the cost of manual bookkeeping.

8 Analogies

1. **City road networks** — intersections are vertices, roads are edges, and one-way streets are directed edges. A GPS shortest-path query is exactly a weighted-graph shortest-path algorithm.
2. **Airline flight maps** — airports are vertices, direct flights are edges weighted by cost or duration. Multi-leg itineraries are paths through the graph.
3. **Social connections** — people are vertices, friendships are undirected edges (mutual), while “follows” on a platform like X are directed edges (one-way).

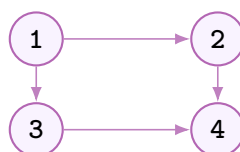
9 Static ASCII Diagrams

Undirected Graph



Edges: 1-2, 1-3, 1-4, 2-4, 3-4

Directed Graph



Edges: $1 \rightarrow 2$, $1 \rightarrow 3$, $2 \rightarrow 4$, $3 \rightarrow 4$

10 BFS Step-by-Step Walkthrough

Consider the undirected graph above, starting BFS from vertex 1.

```

Step 0: queue = [1]           visited = {1}
Step 1: dequeue 1, visit neighbors {2, 3, 4}
       queue = [2, 3, 4]     visited = {1, 2, 3, 4}
Step 2: dequeue 2, neighbors {1, 4} already visited
       queue = [3, 4]
Step 3: dequeue 3, neighbors {1, 4} already visited
       queue = [4]
Step 4: dequeue 4, neighbors {1, 2, 3} already visited
       queue = []           -- BFS complete

```

Output order: 1 2 3 4

11 Complexity Intuition — $O(V + E)$ vs $O(V^2)$

Adjacency List Traversal

For each of the V vertices, you walk exactly the edges attached to it — no more, no less. Summed across all vertices, the total work is proportional to the total edge count E , plus the V constant-time vertex visits. Hence $O(V + E)$.

```

for each vertex v in V:      <- V iterations
    for each edge (v, u):    <- total edges visited across ALL v = E
        process(u)
Total work = V + E

```

Adjacency Matrix Traversal

To find the neighbors of a single vertex, you must scan an entire row of length V , even if that vertex has only one real neighbor — because the matrix stores a 0/1 flag for every *possible* edge, not just the ones that exist.

```

for each vertex v in V:      <- V iterations
    for each possible target t in V: <- V iterations, REGARDLESS of edges
        if adj[v][t]: process(t)
Total work = V * V = V^2

```

Pro Tip

This is why **sparse graphs** ($E \approx V$) strongly favor adjacency lists, while **dense graphs** ($E \approx V^2$) make the matrix's simplicity worth its cost.

12 Complexity Reference Table

Operation	Adjacency List	Adjacency Matrix
Add Vertex	$O(1)$	$O(V^2)$ (must resize matrix)
Add Edge	$O(1)$	$O(1)$
Remove Vertex	$O(V + E)$	$O(V^2)$
Remove Edge	$O(\text{degree}(v))$	$O(1)$
Query edge (u, v) ?	$O(\text{degree}(u))$	$O(1)$
Traverse all edges	$O(V + E)$	$O(V^2)$
Space	$O(V + E)$	$O(V^2)$

Table 3: Complexity comparison: adjacency list vs adjacency matrix

13 Golden Rules

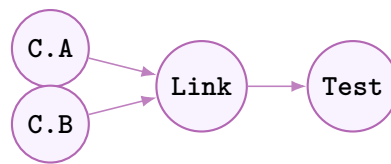
1. **Always track visited nodes** — before recursing/enqueueing, not after processing.
2. **Use adjacency lists for sparse graphs, matrices for dense graphs** — the crossover point is roughly when E approaches V^2 .
3. **Match the representation to the query pattern** — if you need frequent $O(1)$ “is there an edge?” checks, matrices win despite the memory cost.
4. **Free every malloc'd node exactly once** — for adjacency lists, that means walking and freeing every linked list, then freeing the array of list heads.

14 Algorithmic Patterns

14.1 Topological Sort

When to use: ordering tasks with dependencies (build systems, course prerequisites, instruction scheduling) — requires a **Directed Acyclic Graph (DAG)**.

Text visual:



Valid order: Compile A, Compile B, Link, Test

C Code (Kahn's Algorithm, Using In-degree Counting)

```

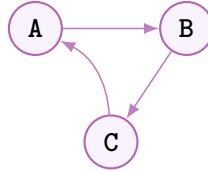
1  void topologicalSort(GraphList *g, int V) {
2      int *inDegree = calloc(V, sizeof(int));
3      for (int v = 0; v < V; v++) {
4          for (Node *cur = g->adjList[v]; cur != NULL; cur = cur->next)
5              inDegree[cur->dest]++;
6      }
7
8      int *queue = malloc(V * sizeof(int));
9      int front = 0, back = 0;
10     for (int v = 0; v < V; v++)
11         if (inDegree[v] == 0) queue[back++] = v;
12
13     int *order = malloc(V * sizeof(int));
14     int idx = 0;
15
16     while (front < back) {
17         int v = queue[front++];
18         order[idx++] = v;
19         for (Node *cur = g->adjList[v]; cur != NULL; cur = cur->next) {
20             if (--inDegree[cur->dest] == 0)
21                 queue[back++] = cur->dest;
22         }
23     }
24
25     if (idx != V) {
26         printf("Graph has a cycle -- no valid topological order.\n");
27     } else {
28         printf("Topological order: ");
29         for (int i = 0; i < idx; i++) printf("%d ", order[i]);
30         printf("\n");
31     }
32
33     free(inDegree);
34     free(queue);
35     free(order);
36 }
    
```

Variations: DFS-based topological sort (push to a stack on post-order finish); lexicographically smallest topological order (use a min-heap instead of a plain queue).

14.2 Cycle Detection

When to use: validating a DAG before running topological sort; detecting deadlocks in resource-allocation graphs; verifying dependency graphs (e.g., npm/cargo) are well-formed.

Text visual:



Cycle: $A \rightarrow B \rightarrow C \rightarrow A$

C Code (DFS with Three-State Color Array)

```

1  int hasCycleUtil(GraphList *g, int v, int *color) {
2      color[v] = 1; /* in progress (on current DFS stack) */
3      for (Node *cur = g->adjList[v]; cur != NULL; cur = cur->next) {
4          if (color[cur->dest] == 1) return 1; /* back edge → cycle */
5          if (color[cur->dest] == 0 && hasCycleUtil(g, cur->dest, color))
6              return 1;
7      }
8      color[v] = 2; /* fully processed */
9      return 0;
10 }
11
12 int hasCycle(GraphList *g, int V) {
13     int *color = calloc(V, sizeof(int));
14     for (int v = 0; v < V; v++) {
15         if (color[v] == 0 && hasCycleUtil(g, v, color)) {
16             free(color);
17             return 1;
18         }
19     }
20     free(color);
21     return 0;
22 }

```

Pro Tip

For undirected graphs, a cycle is detected differently — via a “parent” check during DFS instead of a three-color scheme, since an edge back to the immediate parent is not a cycle.

15 Full Code Example — Adjacency List Graph From Scratch

The following is a complete, compilable C program (`gcc -Wall -Wextra -std=c11 graph.c -o graph`) that builds an adjacency-list graph, runs BFS, and properly tears down all allocated memory.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  typedef struct Node {
5      int dest;
6      struct Node *next;
7  } Node;
8
9  typedef struct {
10     int V;
11     Node **adjList;
12 } GraphList;
13
14 /* --- Construction --- */
15
16 Node *createNode(int dest) {
17     Node *newNode = malloc(sizeof(Node));
18     if (!newNode) { fprintf(stderr, "malloc failed\n"); exit(1); }
19     newNode->dest = dest;
20     newNode->next = NULL;
21     return newNode;
22 }
23
24 GraphList *createGraph(int V) {
25     GraphList *g = malloc(sizeof(GraphList));
26     g->V = V;
27     g->adjList = malloc(V * sizeof(Node *));
28     for (int i = 0; i < V; i++)
29         g->adjList[i] = NULL;
30     return g;
31 }
32
33 /* Undirected edge: add both directions */
34 void addEdge(GraphList *g, int src, int dest) {
35     Node *n1 = createNode(dest);
36     n1->next = g->adjList[src];
37     g->adjList[src] = n1;
38
39     Node *n2 = createNode(src);
40     n2->next = g->adjList[dest];
41     g->adjList[dest] = n2;
42 }
```

```

43
44  /* --- Traversal --- */
45
46  void bfs(GraphList *g, int start) {
47      int *visited = calloc(g→V, sizeof(int));
48      int *queue = malloc(g→V * sizeof(int));
49      int front = 0, back = 0;
50
51      visited[start] = 1;
52      queue[back++] = start;
53
54      printf("BFS from %d: ", start);
55      while (front < back) {
56          int v = queue[front++];
57          printf("%d ", v);
58          for (Node *cur = g→adjList[v]; cur ≠ NULL; cur = cur→next) {
59              if (!visited[cur→dest]) {
60                  visited[cur→dest] = 1;
61                  queue[back++] = cur→dest;
62              }
63          }
64      }
65      printf("\n");
66
67      free(visited);
68      free(queue);
69  }
70
71  /* --- Teardown --- */
72
73  void freeGraph(GraphList *g) {
74      for (int i = 0; i < g→V; i++) {
75          Node *cur = g→adjList[i];
76          while (cur ≠ NULL) {
77              Node *tmp = cur;
78              cur = cur→next;
79              free(tmp);
80          }
81      }
82      free(g→adjList);
83      free(g);
84  }
85
86  int main(void) {
87      GraphList *g = createGraph(5);
88      addEdge(g, 0, 1);
89      addEdge(g, 0, 4);
90      addEdge(g, 1, 2);
91      addEdge(g, 1, 3);
92      addEdge(g, 1, 4);
93      addEdge(g, 2, 3);

```

```
94     addEdge(g, 3, 4);
95
96     bfs(g, 0);
97
98     freeGraph(g);
99     return 0;
100 }
```

Expected output: BFS from 0: 0 4 1 3 2 (neighbors are prepended to each list, so they are visited in last-added-first order).

Pro Tip

Every `malloc` (nodes, the `adjList` array, the graph struct, the `visited` array, the queue) has a matching `free` — the program is Valgrind-clean with zero leaks.

16 Bug Scenarios & Fixes

16.1 Bug 1: Infinite Loop from Missing visited Flag

Wrong code:

```
1 void dfs(GraphList *g, int v) {
2     printf("%d ", v);
3     for (Node *cur = g->adjList[v]; cur != NULL; cur = cur->next)
4         dfs(g, cur->dest); /* no visited check */
5 }
```

What breaks: any cycle causes unbounded recursion → stack overflow / segfault.

Correct code:

```
1 void dfs(GraphList *g, int v, int *visited) {
2     visited[v] = 1;
3     printf("%d ", v);
4     for (Node *cur = g->adjList[v]; cur != NULL; cur = cur->next)
5         if (!visited[cur->dest])
6             dfs(g, cur->dest, visited);
7 }
```

Key lesson: a graph is not a tree — always assume cycles are possible and guard against them explicitly.

16.2 Bug 2: Segfault on Disconnected Components

Wrong code:

```
1  bfs(g, 0); /* only traverses the component containing vertex 0 */
2  processAllVertices(g, visited); /* uninitialized visited[] for unreached
   vertices */
```

What breaks: if the caller assumes single-BFS-call coverage of the whole graph, vertices in other components are silently skipped.

Correct code:

```
1  int *visited = calloc(g->V, sizeof(int)); /* calloc zero-initializes */
2  for (int v = 0; v < g->V; v++) {
3      if (!visited[v]) bfs(g, v, visited); /* loop over ALL vertices */
4  }
```

Key lesson: never assume a graph is fully connected — always iterate over every vertex and start a fresh traversal for each unvisited one.

16.3 Bug 3: Memory Leaks in Adjacency Lists

Wrong code:

```
1  void freeGraph(GraphList *g) {
2      free(g->adjList); /* BUG: frees the pointer array, but not the nodes */
3      free(g);
4  }
```

What breaks: every Node allocated by createNode is orphaned — Valgrind reports “definitely lost” for every edge.

Correct code:

```
1  void freeGraph(GraphList *g) {
2      for (int i = 0; i < g->V; i++) {
3          Node *cur = g->adjList[i];
4          while (cur != NULL) {
5              Node *tmp = cur;
6              cur = cur->next;
7              free(tmp); /* free each node individually first */
8          }
9      }
10     free(g->adjList); /* then the array of heads */
11     free(g); /* then the struct itself */
```

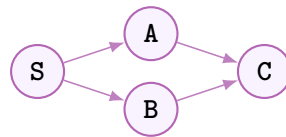


```
12 }
```

Key lesson: teardown order must mirror the allocation hierarchy in reverse — free the deepest/most-nested allocations first.

17 Graph Variants

17.1 Directed Acyclic Graph (DAG)



Characteristics: directed edges, no cycles, supports a valid topological ordering. Used for build systems, scheduling, version-control commit history. The DAG property is enforced by *how* edges are added, not by the struct shape itself.

17.2 Weighted Graph

Struct modification (add a `weight` field to `Node`):

```

1  typedef struct Node {
2      int dest;
3      int weight;      /* NEW: edge weight */
4      struct Node *next;
5  } Node;

```

Use case: shortest-path (Dijkstra), minimum spanning tree (Prim's/Kruskal's), network-flow problems.

17.3 Bipartite Graph

Every edge connects a vertex in Set A to a vertex in Set B — never within the same set. Used for matching problems (job assignment, stable marriage), recommendation systems (users ↔ items).

Struct modification:

```

1  typedef struct {
2      int V;
3      Node **adjList;
4      int *color; /* NEW: 0/1 coloring array to verify/track the bipartition */
5  } BipartiteGraph;

```

Bipartiteness is a *property* verified via 2-coloring, not a structural change to the edge representation.

18 Conceptual Deep-Dive: Memory Layout and Cache Locality

The adjacency matrix and adjacency list are not just two APIs for the same idea — they represent fundamentally different memory-layout philosophies, and that difference has real performance consequences on modern CPUs.

Deep Dive

Contiguous memory (matrix): a 2D array in C is stored as one contiguous block of $V \times V$ integers in row-major order. When you scan row i (`adj[i][0]`, `adj[i][1]`, ..., `adj[i][V-1]`), each access is to the next memory address, which the CPU's prefetcher and cache lines handle extremely efficiently — a single cache-line fetch can satisfy many consecutive accesses. This is why, despite its $O(V^2)$ asymptotic cost, a matrix can outperform a list on **dense** graphs in wall-clock time: the constant factor is tiny due to cache locality.

Scattered memory (list): each `Node` is a separately `malloc`'d block, and `malloc` gives no guarantee about where in the heap that memory lands. Walking a linked list means following a `next` pointer to a potentially distant memory address on every step — a **cache miss** on essentially every node access in the worst case. This is why, even though the adjacency list is asymptotically superior ($O(V + E)$) for sparse graphs, its real-world performance can suffer from pointer-chasing overhead.

Practical implication: for performance-critical, sparse, static graphs, professional code-bases often prefer CSR-style flat-array adjacency lists (Compressed Sparse Row) over pointer-based linked lists — combining $O(V + E)$ traversal cost with matrix-like cache locality.

19 Allocation Mechanics: Stack vs Heap

In C, the **stack** holds function-local variables with sizes known at compile time — fixed-size arrays, `int`, pointers themselves (not what they point to). The **heap** holds anything allocated with `malloc/calloc`, whose size can be determined at *runtime*.

Graphs almost always require heap allocation because:

1. **The number of vertices (V) is not known at compile time** — a fixed-size stack array (`int adj[100][100]`) either wastes memory or hard-caps your graph size. `malloc`'d arrays sized exactly to V avoid both problems.
2. **The number of edges (E) is dynamic and variable per vertex** — a linked-list-based adjacency list grows one `Node` at a time as edges are added, which is only possible with heap allocation; you cannot resize a stack array.

Exact teardown process to avoid leaks, in strict reverse order of allocation:

```
1 void freeGraph(GraphList *g) {  
2     for (int i = 0; i < g->V; i++) { /* 1. free each linked list's nodes */  
3         Node *cur = g->adjList[i];  
4         while (cur != NULL) {  
5             Node *tmp = cur;  
6             cur = cur->next;  
7             free(tmp);  
8         }  
9     }  
10    free(g->adjList); /* 2. free the array of list-head pointers */  
11    free(g);          /* 3. free the graph struct itself */  
12 }
```

Warning

Any deviation from this order — freeing g before its `adjList`, or `adjList` before its nodes — either leaks memory or causes a use-after-free dangling-pointer bug.

20 Problem Ladder

- **Level 1 (Easy):** Given an undirected graph and two vertices u and v , determine whether a path exists between them.
- **Level 2 (Medium):** Given an undirected graph, count the number of disconnected components.
- **Level 3 (Hard):** Given a weighted, directed graph, find the shortest path from a source vertex to all other vertices.

21 Hints

- **Level 1:** A single BFS or DFS from u , checking whether v ends up in the visited set, is sufficient — no need for anything more elaborate.
- **Level 2:** Loop over all vertices; every time you find an unvisited one, that's a new component — start a fresh traversal from it and increment a counter.
- **Level 3:** This is a classic use case for a priority-queue-driven greedy algorithm that always expands the currently-closest unvisited vertex (Dijkstra's algorithm).

22 Complexity Table (Reference Copy)

Operation	Adjacency List	Adjacency Matrix
Add Vertex	$O(1)$	$O(V^2)$
Add Edge	$O(1)$	$O(1)$
Remove Vertex	$O(V + E)$	$O(V^2)$
Remove Edge	$O(\text{degree}(v))$	$O(1)$
Query edge (u, v) ?	$O(\text{degree}(u))$	$O(1)$
Traverse all edges	$O(V + E)$	$O(V^2)$
Space	$O(V + E)$	$O(V^2)$

Table 4: Complexity reference

23 Pattern Cheat Sheet

Pattern	Complexity	Space
BFS (shortest path in unweighted)	$O(V + E)$	$O(V)$
DFS (cycle detection, connectivity)	$O(V + E)$	$O(V)$
Topological Sort (Kahn's)	$O(V + E)$	$O(V)$
Cycle Detection (directed)	$O(V + E)$	$O(V)$
Cycle Detection (undirected)	$O(V + E)$	$O(V)$
Dijkstra's Algorithm	$O((V + E) \log V)$	$O(V)$

Table 5: Algorithm pattern reference

24 Competitive Programming Platform Tips

- **I/O speedups for large inputs:** competitive programming judges (Codeforces, LeetCode's C track, ICPC) often feed graphs with V, E up to 10^5 - 10^6 . Default `scanf/printf` are usually fine in C (unlike C++'s `cin/cout`), but for very large inputs, prefer buffered custom fast-readers (reading the whole input via `fread` into a buffer and parsing integers manually).
- **1-based vs 0-based indexing:** most competitive judges present vertices as 1-indexed (1 to V), while C arrays are naturally 0-indexed. Decide once, up front, whether to allocate arrays of size $V + 1$ and ignore index 0, or subtract 1 from every input vertex on read. Mixing the two conventions mid-program is the single most common source of off-by-one segfaults in competitive C graph code.
- **Staying within $O(V + E)$:** always default to adjacency lists (or CSR arrays) for competitive inputs, since V and E are typically large enough that an $O(V^2)$ matrix will time out (TLE) even though it would pass for small hand-written test cases.

25 Common Mistakes — CP Checklist

- ☐ Clear (or reallocate) the `visited` array between test cases — a stale array from a previous test case silently produces wrong output on the next one.
- ☐ Check for multiple edges between the same pair of vertices (multigraphs) — some problems require counting or deduplicating them explicitly.
- ☐ Watch for self-loops (an edge from a vertex to itself) — decide up front whether your algorithm should count them as valid degree contributions.
- ☐ Confirm whether the graph is directed or undirected before choosing whether `addEdge` inserts one or two `Nodes`.
- ☐ Free (or otherwise reset) all dynamically allocated per-test-case memory — memory not freed between test cases can silently exhaust the judge's memory limit on graphs with many test cases.

26 Extended Debugging — Self-Loops and Isolated Vertices

26.1 Bug: Self-loop Mishandling

Wrong: `addEdge(g, 3, 3)` inserted naively creates a `Node` where `dest == 3` in vertex 3's own list. A naive BFS/DFS visited-check still works — but a naive **degree-counting** function that doesn't special-case self-loops will double-count the self-loop's contribution to degree (since some definitions count a self-loop as +2 to degree, others as +1).

Fix: explicitly decide and document your self-loop degree convention, and special-case it:

```
1  int degree(GraphList *g, int v) {
2      int d = 0;
3      for (Node *cur = g->adjList[v]; cur != NULL; cur = cur->next) {
4          d += (cur->dest == v) ? 2 : 1; /* explicit: self-loop counts as 2 */
5      }
6      return d;
7  }
```

26.2 Bug: Isolated Vertices Skipped Entirely

Wrong: code that only allocates `adjList` entries for vertices that appear in at least one `addEdge` call will crash or behave incorrectly when a vertex with zero edges is queried, because `g->adjList[v]` for that vertex was never initialized to `NULL`.

Fix: always initialize every slot in `adjList` to `NULL` at graph creation time (as shown in `createGraph` in the Full Implementation section), regardless of whether every vertex will eventually receive an edge — this guarantees isolated vertices are represented correctly as “a vertex with an empty list,” not as undefined memory.

Warning

Fix cycle summary: define your conventions (directed vs undirected, self-loop degree counting, isolated-vertex initialization) *before* writing traversal code, and encode those conventions as constants or comments at the top of the file — most graph bugs in practice are convention mismatches, not algorithmic errors.

27 What You've Learned

By working through this guide, you can now:

- Explain why graphs are necessary where arrays and trees fall short, and identify real-world systems (routing, social networks, dependency management, healthcare records) that require graph modeling.
- Implement both adjacency-matrix and adjacency-list representations in raw C, including correct `malloc/free` discipline with zero memory leaks.
- Correctly implement and reason about the `visited`-array discipline that prevents infinite loops on cyclic graphs.
- Run and hand-trace BFS and DFS traversals, and understand precisely why list traversal is $O(V + E)$ while matrix traversal is $O(V^2)$.
- Implement topological sort (Kahn's algorithm) and cycle detection using a three-state DFS coloring scheme.
- Recognize and fix the three most common categories of graph bugs in C: infinite loops, disconnected-component segfaults, and adjacency-list memory leaks.
- Model DAGs, weighted graphs, and bipartite graphs with the appropriate struct modifications.
- Reason about cache locality trade-offs between contiguous (matrix) and scattered (list) memory layouts.
- Apply competitive-programming-specific optimizations: fast I/O, consistent indexing conventions, and per-test-case memory hygiene.

28 Forward-Looking Statement

The next logical topics to pursue from here are:

- **Weighted shortest-path algorithms** — Dijkstra's algorithm (non-negative weights) and Bellman-Ford (handles negative weights, detects negative cycles).
- **Minimum Spanning Trees** — Prim's and Kruskal's algorithms for building a minimum-cost connected subgraph, with Kruskal's requiring a Union-Find (Disjoint Set Union) data structure.
- **Network Flow** — the Ford-Fulkerson / Edmonds-Karp algorithms for max-flow/min-cut problems, foundational to bipartite matching and capacity-constrained routing.
- **Strongly Connected Components** — Tarjan's and Kosaraju's algorithms for decomposing directed graphs into maximal strongly connected subgraphs.

29 Weighted Graph Algorithms — Dijkstra, Prim's, Kruskal's

All three are classic **greedy algorithms** on weighted graphs. Prim's and Kruskal's solve the **Minimum Spanning Tree (MST)** problem; Dijkstra's solves the **Single-Source Shortest Path (SSSP)** problem.

29.1 Prim's Algorithm (Minimum Spanning Tree)

Mathematical Structure

Given a connected, undirected, weighted graph $G = (V, E, w)$ where $w : E \rightarrow \mathbb{R}^{\geq 0}$, find a spanning tree $T = (V, E_T)$, $E_T \subseteq E$, such that $\sum_{e \in E_T} w(e)$ is minimized.

Cut Property: For any cut $(S, V - S)$ of the graph, if an edge e is the unique minimum-weight edge crossing the cut, then e belongs to **every** MST. Prim's algorithm exploits this repeatedly, growing a single tree outward from a root.

Correctness: By induction — at every step E_T is a subset of some MST. The extracted vertex u has the minimum-weight edge crossing the cut $(S, V - S)$, which by the cut property belongs to an MST, so adding it preserves the invariant.

Complexity:

Implementation	Extract-Min	Decrease-Key	Total
Array	$O(V)$	$O(1)$	$O(V^2)$
Binary Heap	$O(\log V)$	$O(\log V)$	$O((V + E) \log V) = O(E \log V)$
Fibonacci Heap	$O(\log V)$	$O(1)$ amortized	$O(E + V \log V)$

Table 6: Prim's algorithm complexity by heap implementation

Best for dense graphs ($E \approx V^2$).

Algorithmic Steps

1. Initialization

- Choose an arbitrary starting vertex $r \in V$.
- Set $S = \{r\}$ (the growing tree set).
- $key[r] = 0$, $key[v] = \infty$ for $v \neq r$; $parent[v] = \text{NIL}$ for all v .
- Insert all vertices into a min-priority queue Q keyed by $key[v]$.

2. Iterative Growth — repeat while $Q \neq \emptyset$:

- (a) Extract $u = \text{Extract-Min}(Q)$; add u to S ; if $parent[u] \neq \text{NIL}$, add $(u, parent[u])$ to E_T .
- (b) For each neighbor v of u with $v \notin S$:
 - If $w(u, v) < key[v]$: set $key[v] = w(u, v)$, $parent[v] = u$, $\text{Decrease-Key}(Q, v, key[v])$.

3. Termination

- $E_T = \{(v, parent[v]) \mid parent[v] \neq \text{NIL}\}$ forms the MST.

C Code (Simple $O(V^2)$ Array-Scan Version)

The code uses the weighted `Node` struct and `addEdge` from the Dijkstra implementation (both directions, same weight).

```

1  #include <limits.h>
2
3  void primMST(GraphList *g) {
4      int V = g->V;
5      int *key = malloc(V * sizeof(int));
6      int *parent = malloc(V * sizeof(int));
7      int *inMST = calloc(V, sizeof(int));
8
9      for (int i = 0; i < V; i++) { key[i] = INT_MAX; parent[i] = -1; }
10     key[0] = 0;
11
12     for (int count = 0; count < V - 1; count++) {
13         int u = -1, min = INT_MAX;
14         for (int v = 0; v < V; v++)
15             if (!inMST[v] && key[v] < min) { min = key[v]; u = v; }
16
17         inMST[u] = 1;
18
19         for (Node *cur = g->adjList[u]; cur != NULL; cur = cur->next) {
20             int v = cur->dest;
21             if (!inMST[v] && cur->weight < key[v]) {

```



```

22         key[v] = cur->weight;
23         parent[v] = u;
24     }
25 }
26 }
27
28 int total = 0;
29 printf("Edge \tWeight\n");
30 for (int i = 1; i < V; i++) {
31     printf("%d - %d \t%d\n", parent[i], i, key[i]);
32     total += key[i];
33 }
34 printf("Total MST weight: %d\n", total);
35
36 free(key); free(parent); free(inMST);
37 }

```

Verified output for a 5-vertex graph with edges:

Edge	Weight
0 - 1	2
1 - 2	3
0 - 3	6
1 - 4	5

Total MST weight: 16

29.2 Kruskal's Algorithm (Minimum Spanning Tree)

Mathematical Structure

Given a connected, undirected, weighted graph $G = (V, E, w)$ with $w : E \rightarrow \mathbb{R}^{\geq 0}$, find a spanning tree $T = (V, E_T)$, $E_T \subseteq E$, such that $\sum_{e \in E_T} w(e)$ is minimized.

Cycle Property + Greedy Choice: Sort edges by weight and greedily add the smallest edge that does **not** form a cycle with previously chosen edges. The cycle property guarantees the maximum-weight edge in any cycle is never in an MST — the safest edges to add are globally minimal ones that respect acyclicity.

Correctness: By induction on the number of edges added using the **cut property**: at each step, the two components being merged define a cut with no edge of smaller weight crossing it (since edges are processed in increasing order), so the chosen edge is safe.

Complexity:

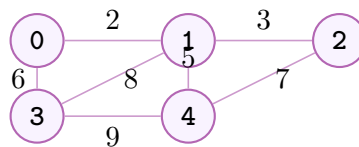
- Sorting edges: $O(E \log E)$
- Union-Find (path compression + union by rank): $O(E \alpha(V))$, inverse Ackermann $\alpha(V) \approx \text{constant}$.

- **Total:** $O(E \log E) = O(E \log V)$.

Best for sparse graphs or edge-list-native inputs.

Algorithmic Steps

1. **Initialization:** $E_T \leftarrow \emptyset$; initialize Union-Find with $\text{MakeSet}(v)$ for each $v \in V$.
2. **Sorting:** sort all edges E in non-decreasing order of weight.
3. **Greedy Selection** — for each edge (u, v) in sorted order:
 - If $\text{Find}(u) \neq \text{Find}(v)$ (different components): add (u, v) to E_T ; $\text{Union}(u, v)$.
 - Else discard (would create a cycle).
4. **Termination:** stop when $|E_T| = |V| - 1$ or all edges processed. E_T is the MST.



Sorted edges by weight: $(0-1,2)$, $(1-2,3)$, $(1-4,5)$, $(0-3,6)$, $(2-4,7)$, $(1-3,8)$, $(3-4,9)$

C Code (Edge List + Union-Find, $O(E \log E)$)

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  typedef struct { int src, dest, weight; } Edge;
5
6  typedef struct {
7      int *parent;
8      int *rank;
9  } DSU;
10
11 DSU *createDSU(int n) {
12     DSU *d = malloc(sizeof(DSU));
13     d->parent = malloc(n * sizeof(int));
14     d->rank = calloc(n, sizeof(int));
15     for (int i = 0; i < n; i++) d->parent[i] = i;
16     return d;
17 }
18
19 int find(DSU *d, int i) {
20     if (d->parent[i] != i)
21         d->parent[i] = find(d, d->parent[i]); /* path compression */
22     return d->parent[i];
23 }
24

```

```

25 void unionSets(DSU *d, int x, int y) {
26     int rx = find(d, x), ry = find(d, y);
27     if (rx == ry) return;
28     if (d->rank[rx] < d->rank[ry]) d->parent[rx] = ry;
29     else if (d->rank[rx] > d->rank[ry]) d->parent[ry] = rx;
30     else { d->parent[ry] = rx; d->rank[rx]++; }
31 }
32
33 void freeDSU(DSU *d) { free(d->parent); free(d->rank); free(d); }
34
35 int cmpEdge(const void *a, const void *b) {
36     return ((Edge *)a)->weight - ((Edge *)b)->weight;
37 }
38
39 void kruskalMST(Edge *edges, int E, int V) {
40     qsort(edges, E, sizeof(Edge), cmpEdge);
41     DSU *d = createDSU(V);
42
43     int total = 0, count = 0;
44     printf("Edge \tWeight\n");
45     for (int i = 0; i < E && count < V - 1; i++) {
46         int x = find(d, edges[i].src);
47         int y = find(d, edges[i].dest);
48         if (x != y) {
49             printf("%d - %d \t%d\n", edges[i].src, edges[i].dest, edges[i].
50                 weight);
51             total += edges[i].weight;
52             unionSets(d, x, y);
53             count++;
54         }
55     }
56     printf("Total MST weight: %d\n", total);
57     freeDSU(d);
58 }
59
60 int main(void) {
61     Edge edges[] = {
62         {0, 1, 2}, {0, 3, 6}, {1, 2, 3},
63         {1, 3, 8}, {1, 4, 5}, {2, 4, 7}, {3, 4, 9}
64     };
65     int E = sizeof(edges) / sizeof(edges[0]);
66
67     kruskalMST(edges, E, 5);
68     return 0;
69 }

```

Verified output:

Edge	Weight
0 - 1	2

1 - 2 3
 1 - 4 5
 0 - 3 6
 Total MST weight: 16

29.3 Dijkstra's Algorithm (Single-Source Shortest Path)

Mathematical Structure

Given a weighted graph $G = (V, E, w)$ with **non-negative** edge weights ($w(e) \geq 0 \forall e$) and a source vertex s , find the shortest-path distance $\delta(s, v)$ for every $v \in V$.

Edge Relaxation + Greedy Finalization: Maintain a tentative distance estimate $d[v] \geq \delta(s, v)$ for all v , and repeatedly finalize the vertex with the smallest tentative distance among unfinalized vertices — this is provably its true shortest distance. Dijkstra fails on negative edge weights since it relies on the greedy assumption that once finalized, a distance cannot be improved later.

Correctness (greedy proof sketch): When u is extracted, $d[u] = \delta(s, u)$. Suppose $d[u] > \delta(s, u)$. On the true shortest path $s \rightarrow u$, let y be the first unfinalized vertex with predecessor $x \in S$. Since x was relaxed:

$$d[y] = \delta(s, y) \leq \delta(s, u) < d[u],$$

contradicting u as the minimum- d vertex in Q . ■

Deep Dive

Structural resemblance to Prim's: both use the “extract-min, relax neighbors” template, differing only in the key update:

- Prim's: $key[v] = \min(key[v], w(u, v))$ — edge weight.
- Dijkstra's: $d[v] = \min(d[v], d[u] + w(u, v))$ — cumulative path weight.

Complexity:

Implementation	Extract-Min	Decrease-Key	Total
Array	$O(V)$	$O(1)$	$O(V^2)$
Binary Heap	$O(\log V)$	$O(\log V)$	$O((V + E) \log V)$
Fibonacci Heap	$O(\log V)$	$O(1)$ amortized	$O(E + V \log V)$

Table 7: Dijkstra's algorithm complexity by heap implementation

Algorithmic Steps

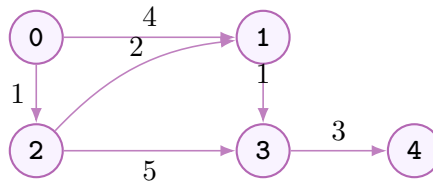
1. **Initialization:** $d[s] = 0$, $d[v] = \infty$ for $v \neq s$; $parent[v] = \text{NIL}$; $S = \emptyset$; insert all vertices into min-priority queue Q keyed by $d[v]$.
2. **Iterative Relaxation** — repeat while $Q \neq \emptyset$:
 - (a) Extract $u = \text{Extract-Min}(Q)$; add u to S .

(b) **Relax** each outgoing edge $(u, v) \in E$:

if $d[u] + w(u, v) < d[v]$: $d[v] \leftarrow d[u] + w(u, v)$, $parent[v] \leftarrow u$, $Decrease\text{-}Key(Q, v, d[v])$

3. **Termination:** $d[v] = \delta(s, v)$ for all reachable v ; shortest paths recoverable via $parent[\cdot]$.

Text visual:



Shortest path $0 \rightarrow 4$: $0 \rightarrow 2 \rightarrow 1 \rightarrow 3 \rightarrow 4$, total cost 7.

C Code (Adjacency List + Binary Min-Heap, $O((V + E) \log V)$)

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <limits.h>
4
5  typedef struct Node {
6      int dest;
7      int weight;
8      struct Node *next;
9  } Node;
10
11 typedef struct {
12     int V;
13     Node **adjList;
14 } GraphList;
15
16 Node *createNode(int dest, int weight) {
17     Node *n = malloc(sizeof(Node));
18     n->dest = dest;
19     n->weight = weight;
20     n->next = NULL;
21     return n;
22 }
23
24 GraphList *createGraph(int V) {
25     GraphList *g = malloc(sizeof(GraphList));
26     g->V = V;
27     g->adjList = malloc(V * sizeof(Node *));
28     for (int i = 0; i < V; i++) g->adjList[i] = NULL;
29     return g;
30 }
31
32 void addEdge(GraphList *g, int src, int dest, int weight) {
33     Node *n1 = createNode(dest, weight);
    
```

```

34     n1→next = g→adjList[src];
35     g→adjList[src] = n1;
36
37     /* Dijkstra on a directed graph: add edge only in src→dest direction */
38 }
39
40 /* --- Binary min-heap keyed on distance --- */
41 typedef struct { int vertex; int dist; } HeapItem;
42
43 typedef struct {
44     HeapItem *items;
45     int *pos;      /* pos[v] = index of vertex v inside items[] */
46     int size;
47 } MinHeap;
48
49 MinHeap *createMinHeap(int capacity) {
50     MinHeap *h = malloc(sizeof(MinHeap));
51     h→items = malloc(capacity * sizeof(HeapItem));
52     h→pos = malloc(capacity * sizeof(int));
53     h→size = 0;
54     return h;
55 }
56
57 void swapHeapItems(HeapItem *a, HeapItem *b) {
58     HeapItem t = *a; *a = *b; *b = t;
59 }
60
61 void minHeapify(MinHeap *h, int idx) {
62     int smallest = idx, left = 2*idx+1, right = 2*idx+2;
63     if (left < h→size && h→items[left].dist < h→items[smallest].dist)
64         smallest = left;
65     if (right < h→size && h→items[right].dist < h→items[smallest].dist)
66         smallest = right;
67     if (smallest ≠ idx) {
68         h→pos[h→items[smallest].vertex] = idx;
69         h→pos[h→items[idx].vertex] = smallest;
70         swapHeapItems(&h→items[smallest], &h→items[idx]);
71         minHeapify(h, smallest);
72     }
73 }
74
75 HeapItem extractMin(MinHeap *h) {
76     HeapItem root = h→items[0];
77     HeapItem last = h→items[h→size - 1];
78     h→items[0] = last;
79     h→pos[last.vertex] = 0;
80     h→pos[root.vertex] = h→size - 1;
81     h→size--;
82     minHeapify(h, 0);
83     return root;
84 }

```

```

85
86 void decreaseKey(MinHeap *h, int vertex, int dist) {
87     int i = h->pos[vertex];
88     h->items[i].dist = dist;
89     while (i && h->items[i].dist < h->items[(i-1)/2].dist) {
90         h->pos[h->items[i].vertex] = (i-1)/2;
91         h->pos[h->items[(i-1)/2].vertex] = i;
92         swapHeapItems(&h->items[i], &h->items[(i-1)/2]);
93         i = (i-1)/2;
94     }
95 }
96
97 int isInHeap(MinHeap *h, int v) { return h->pos[v] < h->size; }
98
99 void dijkstra(GraphList *g, int src) {
100     int V = g->V;
101     int *dist = malloc(V * sizeof(int));
102     MinHeap *h = createMinHeap(V);
103
104     for (int v = 0; v < V; v++) {
105         dist[v] = INT_MAX;
106         h->items[v] = (HeapItem){v, INT_MAX};
107         h->pos[v] = v;
108     }
109     h->size = V;
110
111     dist[src] = 0;
112     decreaseKey(h, src, 0);
113
114     while (h->size > 0) {
115         HeapItem u = extractMin(h);
116         for (Node *cur = g->adjList[u.vertex]; cur != NULL; cur = cur->next) {
117             int v = cur->dest;
118             if (isInHeap(h, v) && dist[u.vertex] != INT_MAX &&
119                 dist[u.vertex] + cur->weight < dist[v]) {
120                 dist[v] = dist[u.vertex] + cur->weight;
121                 decreaseKey(h, v, dist[v]);
122             }
123         }
124     }
125
126     printf("Vertex\tDistance from %d\n", src);
127     for (int v = 0; v < V; v++) printf("%d\t%d\n", v, dist[v]);
128
129     free(dist);
130     free(h->items);
131     free(h->pos);
132     free(h);
133 }
134
135 void freeGraph(GraphList *g) {

```

```
136     for (int i = 0; i < g->V; i++) {
137         Node *cur = g->adjList[i];
138         while (cur) { Node *tmp = cur; cur = cur->next; free(tmp); }
139     }
140     free(g->adjList);
141     free(g);
142 }
143
144 int main(void) {
145     GraphList *g = createGraph(5);
146     addEdge(g, 0, 1, 4);
147     addEdge(g, 0, 2, 1);
148     addEdge(g, 2, 1, 2);
149     addEdge(g, 1, 3, 1);
150     addEdge(g, 2, 3, 5);
151     addEdge(g, 3, 4, 3);
152
153     dijkstra(g, 0);
154
155     freeGraph(g);
156     return 0;
157 }
```

Verified output:

Vertex	Distance from 0
0	0
1	3
2	1
3	4
4	7

Warning

Dijkstra fails on negative edge weights — use **Bellman-Ford** ($O(V \cdot E)$) instead.

29.4 Comparative Summary

Property	Prim's	Kruskal's	Dijkstra's
Solves	MST	MST	SSSP
Graph requirement	Connected, weighted	Weighted (any connectivity $\rightarrow$ MST forest)	Non-negative weights
Greedy criterion	Min edge crossing cut	Min edge, no cycle	Min tentative distance
Core data structure	Priority Queue	Union-Find	Priority Queue
Optimal substructure	Cut Property	Cut/Cycle Property	Optimal subpaths
Time complexity (heap)	$O(E \log V)$	$O(E \log V)$	$O(E \log V)$
Output	Set of edges (tree)	Set of edges (tree)	Distance + parent array

Table 8: Comparative summary of Prim's, Kruskal's, and Dijkstra's algorithms

— End of Handout —